# Actividad 4: Manipulación de Cadenas y de Punteros en C/C++

Sofia Mojica, Gabriel Zacarías, Mateo Roberts y Leonel De Armas

Escuela Técnica 5.° 3.° A-B

Laboratorio de Programación (LPR)

Prof. York Mansilla y Prof. Yamil Ganduglia

15 de octubre de 2026

## Introducción

El presente informe documenta el desarrollo y los fundamentos teóricos de la Actividad 4, enfocada en la programación de bajo nivel mediante la manipulación de la memoria de acceso aleatorio (RAM). A diferencia de los lenguajes de alto nivel, que abstraen la gestión de la memoria, C++ permite interactuar directamente con la arquitectura física del hardware. En este trabajo se analiza la implementación de un sistema de control de accesos que interactúa con celdas de memoria referenciadas por direcciones hexadecimales únicas, utilizando punteros para optimizar el rendimiento del software.

## Desarrollo

### Glosario y Fundamentos de Arquitectura

A continuación se definen los conceptos fundamentales empleados en la actividad.

*Memoria RAM.* Unidad de almacenamiento volátil y temporal donde el sistema operativo asigna bloques de celdas para alojar los datos binarios de los procesos en ejecución.

*Dirección hexadecimal.* Sistema de numeración posicional en base 16 (prefijo 0x) que simplifica la lectura de los bits que componen el mapa de direcciones físicas del bus de datos.

*Puntero (pointer).* Variable escalar cuyo valor codificado corresponde de forma unívoca a la dirección física en la RAM de otra estructura de datos del sistema.

*Operador de dirección (&).* Operador unario que extrae la posición física en el hardware de memoria de cualquier variable asignada por el compilador.

*Desreferenciar (*).* Instrucción de máquina que permite saltar de forma directa a la ubicación física apuntada por un puntero, otorgando privilegios de lectura o mutación sobre el valor original.

### Distribución de Roles y Tareas del Equipo

La Tabla 1 detalla el rol y la tarea asignada a cada integrante del equipo.

**Tabla 1**

*Roles y Tareas Asignadas por Integrante*

| Integrante | Rol/Tarea asignada |
|---|---|
| Sofia Mojica | Implementación de la lógica condicional de control de acceso por edad y manejo de lógica de punteros. |
| Gabriel Zacarías | Desarrollo del código de mapeo de direcciones hexadecimales (0x) y desreferenciación directa en la RAM. |
| Mateo Roberts | Estructuración del repositorio base cadenaspunteros, control de versiones en Git y pruebas de terminal. |
| Leonel De Armas | Diseño manual/digital del esquema físico de la RAM, resolución del cuestionario e informe en APA v7. |

## Estructura del Repositorio del Proyecto

El repositorio *cadenaspunteros* se organiza en carpetas exclusivas para la documentación, el código fuente y las evidencias de ejecución, como se muestra en la Figura 1.

**Figura 1**

*Estructura de Carpetas del Repositorio cadenaspunteros*

```
cadenaspunteros/
|-- .gitignore      <-- Hace que Git ignore carpetas de configuracion (.vscode)
|-- LICENSE         <-- Licencia MIT del proyecto (uso escolar)
|-- README.md       <-- Explicacion del uso de punteros en la RAM
|-- docs/           <-- Carpeta exclusiva para documentacion
|   `-- InformeProyectoCADENASPUNTEROS.pdf   <-- Entregable (informe en APA v7)
|-- src/            <-- Codigos fuente del proyecto
|   `-- main.cpp    <-- Codigo fuente modular en C++
`-- capturas/       <-- Evidencias locales (imagenes .png)
    |-- ejecucion_punteros.png   <-- Terminal con direcciones hexadecimales
    `-- esquema_ram.png          <-- Diagrama de memoria hecho a mano
```

*Nota.* Elaboración propia.

## Cuestionario Técnico y Análisis de Memoria

### *Ventaja de modificar una variable a través de un puntero*

Al realizar una llamada común a una función, el sistema operativo realiza una copia de la variable en el segmento de la pila (*stack*), lo que duplica el consumo de espacio en la memoria RAM y añade sobrecarga a la CPU al copiar los datos. Al utilizar un puntero, el

programa solo transfiere una dirección de 4 u 8 bytes (dependiendo de la arquitectura), modificando los datos directamente en su casillero original. Esto agiliza el rendimiento de los sistemas de software críticos y optimiza el uso del hardware.

***Cambio de las direcciones de memoria en cada ejecución***

Esto ocurre debido a una medida de seguridad de los sistemas operativos modernos conocida como ASLR (*Address Space Layout Randomization*, aleatorización del diseño del espacio de direcciones). Cada vez que se lanza el archivo ejecutable, el sistema operativo asigna un bloque dinámico de celdas en la RAM que se encuentren libres en ese instante. Como la memoria se comparte con otros programas y procesos en segundo plano, la dirección física exacta (0x...) cambiará inevitablemente en cada ciclo de ejecución.

***Peligro de usar un puntero a una dirección inválida***

Si un puntero no se inicializa (puntero salvaje) o apunta a un bloque de memoria que ya fue liberado, el programa intentará invadir celdas protegidas del sistema operativo o de otros procesos de software. Cuando la CPU detecta esta anomalía, el sistema aborta la ejecución del programa de forma inmediata para proteger su integridad, provocando un error crítico conocido como *segmentation fault* (fallo de segmentación) o cuelgue del programa.

## Conclusión

El estudio práctico de la arquitectura de la memoria RAM a través del uso de punteros constituye un pilar indispensable para la formación de técnicos especialistas en desarrollo de software. Comprender los mecanismos de direccionamiento hexadecimal y el uso de los operadores de bajo nivel (& y *) provee las herramientas necesarias para escribir algoritmos eficientes en el uso de memoria, sentando bases sólidas para el manejo avanzado de estructuras de datos dinámicas en los próximos años del ciclo técnico profesional. El código completo y las evidencias de ejecución se encuentran disponibles en el repositorio del proyecto (Mojica et al., 2026).

## Referencias

Mojica, S., Zacarías, G., Roberts, M., & De Armas, L. (2026). *cadenaspunteros* [Repositorio de código]. GitHub. https://github.com/matuteXEN6/cadenaspunteros.git